

Politechnika Warszawska
Wydział Matematyki i Nauk Informacyjnych

Zaawansowane Programowanie Obiektowe i Funkcyjne

Generator Danych Osobowych

DOKUMENTACJA PROJEKTU

Autorzy:

Szymon Domański
Krzysztof Dąbrowski
Gustaw Gabrylewski

Rok akademicki 2025/2026

26 stycznia 2026

Spis treści

1	Opis Wstępny	2
1.1	Kontekst biznesowy	2
1.2	Przedmiot projektu	2
1.3	Grupa docelowa	2
2	Założenia projektowe	2
2.1	Wymagania funkcjonalne	2
2.2	Wymagania niefunkcjonalne	3
3	Schemat systemu	3
3.1	Opis warstw	3
4	Wykorzystane technologie	3
5	Opis implementacji	4
5.1	Generowanie numerów identyfikacyjnych	4
5.2	Dynamiczny eksport danych	4
5.3	Moduł Zaszumiania (Data Corruptor)	4
6	Instrukcja wdrożeniowa	4
6.1	Wymagania	5
6.2	Uruchomienie w IntelliJ IDEA	5
7	Instrukcja użytkownika	5
8	Podsumowanie i wnioski	5

1 Opis Wstępny

1.1 Kontekst biznesowy

W procesie tworzenia oprogramowania, a w szczególności na etapie testów, kluczowym zagadnieniem jest posiadanie odpowiednich zbiorów danych testowych. Używanie rzeczywistych danych osobowych klientów jest prawnie zabronione (RODO/GDPR) i niesie ryzyko naruszenia prywatności. Z kolei ręczne tworzenie setek rekordów jest czasochłonne i podatne na błędy ludzkie.

1.2 Przedmiot projektu

Celem projektu jest stworzenie aplikacji desktopowej, która automatyzuje proces tworzenia syntetycznych, ale realistycznych danych obywateli Polski. Aplikacja ma symulować dane tożsamości, teleadresowe oraz kontaktowe, zachowując logiczną i geograficzną spójność.

1.3 Grupa docelowa

Narzędzie dedykowane jest:

- **Testerom oprogramowania:** do zasilania środowisk testowych.
- **Programistom:** do weryfikacji walidatorów formularzy (PESEL, NIP).
- **Analitykom danych:** do testów wydajnościowych baz danych (Big Data).

2 Założenia projektowe

2.1 Wymagania funkcjonalne

1. **Generowanie tożsamości:** Aplikacja musi generować imię, nazwisko (zgodne z płcią), datę urodzenia oraz płeć.
2. **Numery identyfikacyjne:** Generowanie poprawnych algorytmicznie numerów PESEL, NIP oraz numeru Dowodu Osobistego (z prawidłowymi sumami kontrolnymi).
3. **Spójność geograficzna:** Miasto musi być powiązane z właściwym województwem, a kod pocztowy musi odpowiadać danemu regionowi.
4. **Konfiguracja:** Użytkownik może określić liczbę rekordów, zakres wieku oraz płeć generowanych osób.
5. **Selekcja danych:** Możliwość wyboru konkretnych kolumn do wygenerowania.
6. **Eksport danych:** Zapis wyników do formatów CSV, JSON oraz SQL.
7. **Zaszumianie danych:** Możliwość celowego wprowadzania błędów z regulowanym prawdopodobieństwem (0-50%).

2.2 Wymagania niefunkcjonalne

1. **Wydajność:** Czas generowania 10 000 rekordów nie powinien przekraczać kilku sekund.
2. **Realizm:** Imiona, nazwiska i miasta powinny być losowane na podstawie wag popularności w Polsce.
3. **Użyteczność:** Przejrzysty interfejs graficzny (GUI) z podglądem danych w formie tabeli.
4. **Rozszerzalność:** Architektura kodu powinna umożliwiać łatwe dodanie nowych formatów eksportu.

3 Schemat systemu

Projekt został zrealizowany w oparciu o wzorzec architektoniczny **MVC (Model-View-Controller)**, co zapewnia separację logiki biznesowej od warstwy prezentacji.

3.1 Opis warstw

- **Model (Pakiet model):** Klasy reprezentujące strukturę danych pojedynczego rekordu.
- **Logika Biznesowa (Pakiet logic):**
 - **DictionaryLoader:** Odpowiada za wczytanie plików CSV do pamięci RAM przy starcie aplikacji.
 - **PersonGenerator:** Koordynuje proces tworzenia osoby.
 - **WeightedDictionary:** Implementacja losowania ważonego (popularne cechy wypadają częściej).
 - **DataCorruptor:** Moduł wprowadzający anomalie do poprawnych obiektów.
- **Widok (Pakiet gui):** Klasa **MainView** dziedzicząca po **JFrame**. Odpowiada za układ okna, tabelę wyników (**JTable**), panel boczny z formularzem i suwakami.
- **Kontroler (Pakiet gui):** Klasa **MainController** pośredniczy między widokiem a logiką. Obsługuje zdarzenia i waliduje dane.
- **Eksport (Pakiet export):** Interfejs **DataExporter** i jego implementacje (**CsvExporter**, **JsonExporter**, **SqlExporter**).

4 Wykorzystane technologie

- **Język programowania:** Java (JDK 8+).
- **Biblioteka GUI:** Java Swing.
- **Obsługa I/O:** `java.io`.
- **Środowisko programistyczne:** IntelliJ IDEA.
- **System kontroli wersji:** Git.

5 Opis implementacji

5.1 Generowanie numerów identyfikacyjnych

Zaimplementowano oficjalne algorytmy walidacji dla polskich dokumentów.

- **PESEL:** Obliczany na podstawie daty urodzenia i płci. Uwzględnia przesunięcie miesiąca dla osób urodzonych po roku 2000 i poprawną sumę kontrolną.
- **NIP:** Uwzględnia wagi i operację modulo 11 do utworzenia sumy kontrolnej.
- **Numer Dowodu Osobistego:** Odpowiedni Format (3 litery + 6 cyfr) z poprawną sumą kontrolną.

5.2 Dynamiczny eksport danych

Klasy eksportujące przyjmują listę kolumn wybranych przez użytkownika w GUI. W przypadku SQL dynamicznie budowane jest zapytanie `CREATE TABLE` oraz `INSERT INTO`, zawierające tylko zaznaczone pola.

5.3 Moduł Zaszumiania (Data Corruptor)

Kluczowy element dla testerów. Poniżej przedstawiono fragment implementacji metody wprowadzającej błędy:

- **Podróżnik w czasie:** Ustawienie daty urodzenia na dzień jutrzejszy lub dalszą przyszłość (symulacja błędu logicznego w systemach, np. ujemny wiek).
- **Czeski błąd:** Losowa zamiana miejscami dwóch sąsiadujących liter w nazwisku (np. *Kowalsik* zamiast *Kowalski*) – testowanie algorytmów wyszukiwania rozmytego (fuzzy search).
- **Białe znaki:** Doklejenie spacji na początku lub na końcu imienia – błąd często niewidoczny wizualnie, a powodujący problemy przy porównywaniu ciągów znaków w bazach danych.
- **Uszkodzenie PESEL:** Wstrzyknięcie litery 'X' w losowe miejsce ciągu cyfr, co narusza format numeryczny i sumę kontrolną.
- **Błędy formatu:** Usunięcie znaku @ z adresu e-mail oraz celowe skracanie numerów telefonów (ucinanie ostatniej cyfry).
- **Błędy wielkości liter:** Zamiana pierwszej litery imienia na małą.
- **Brak danych:** Celowe usuwanie wartości (ustawianie null) dla pól opcjonalnych, takich jak numer domu czy nazwisko.

6 Instrukcja wdrożeniowa

Aby uruchomić aplikację na nowym stanowisku, należy spełnić następujące wymagania.

6.1 Wymagania

- Zainstalowane środowisko Java Runtime Environment (JRE) lub JDK w wersji 8 lub nowszej.
- Pobrany kod źródłowy projektu.

6.2 Uruchomienie w IntelliJ IDEA

1. Otwórz projekt w IDE.
2. Skompiluj i uruchom klasę `src/main/java/generator/App.java`.

7 Instrukcja użytkownika

1. **Konfiguracja:** W panelu bocznym po lewej stronie wpisz liczbę osób do wygenerowania oraz pożądane zakresy wieku i płci.
2. **Wybór danych:** Zaznacz checkboxy przy danych, które Cię interesują.
3. **Zaszumianie:** Jeśli chcesz przetestować odporność systemu na błędy, przesun suwak "*Poziom błędów*" w prawo (np. na 10%).
4. **Generowanie:** Kliknij przycisk **GENERUJ DANE**. Wyniki pojawią się w tabeli.
5. **Raport statystyczny:** Po wygenerowaniu, automatycznie otwiera się okno z analizą statystyczną danych. Po zamknięciu okna, aby móc ponownie zobaczyć statystyki, kliknij przycisk **Pokaż ostatni raport**.
6. **Eksport:** Kliknij przycisk odpowiadający formatowi (CSV, JSON, SQL), aby zapisać plik na dysku.

8 Podsumowanie i wnioski

Projekt "Generator Danych Osobowych" został zrealizowany zgodnie z założeniami. Udało się stworzyć narzędzie, które jest nie tylko generatorem losowym, ale systemem dbającym o spójność logiczną i geograficzną danych.

Wnioski z realizacji:

- Zastosowanie wzorca MVC znacznie ułatwiło rozbudowę aplikacji o nowe funkcje (np. dodanie suwaka zaszumiania nie wymagało zmian w logice generowania osoby).
- Użycie struktur drzewiastych (`TreeMap`) do losowania ważonego pozwoliło zachować wysoką wydajność nawet przy bardzo dużych słownikach nazwisk.
- Dynamiczny eksport danych czyni narzędzie uniwersalnym dla różnych grup odbiorców.